

Lesson 12 - Homework

File Operations - Fayllarla işləmə

Bu homework-da əvvəlki **Book Management System** layihəsini davam etdirəcəksiniz.

Əsas məqsəd kitab məlumatlarını faylda saxlamaq və proqram yenidən başladıldıqda həmin məlumatları geri oxumaqdır.

Əvvəlcə aşağıdakı xatırlatma hissəsini oxuyun. Nümunələri ayrıca sınaq faylında işlədə bilərsiniz. Daha sonra öyrəndiklərinizi kitab layihənizə tətbiq edin.

Dərstdən qısa xatırlatma

1. Fayla niyə ehtiyacımız var?

Belə bir list təsəvvür edin:

```
names = ["Ayxan", "Tural"]
```

Proqram işləyərkən bu list-dən istifadə edə bilərik. Amma proqramın işlədiyi müddətdə əlavə etdiyimiz məlumatlar proqram bağlandıqdan sonra avtomatik saxlanılmır.

Növbəti run əvvəlki run-ın yaddaşını davam etdirmir. Kod yenidən işləyir.

Məlumatların yadda saxlanması üçün onları fayla yazmalıyıq.

Məsələn, mətn faylında belə məlumat saxlaya bilərik:

```
Ayxan  
Tural
```

Sonra proqram yenidən başladıldıqda həmin faylı oxuya bilərik.

Yadda saxlayın:

- List məlumatı proqramın yaddaşında saxlayır.
- Fayl məlumatın proqram bağlandıqdan sonra da qalmasına imkan verir.
- List dəyişəndə fayl avtomatik dəyişmir. Bunun üçün ayrıca yazma əməliyyatı etməliyik.

2. İlk mətn faylımızı yaradaq

Nümunələri yoxlamaq üçün ayrıca Python faylı yaradın:

```
file_demo.py
```

İçində yazın:

```
with open("participants.txt", "w", encoding="utf-8") as file:  
    file.write("Ayxan\n")  
    file.write("Tural\n")
```

Proqramı run edin.

participants.txt faylının məzmunu belə olacaq:

```
Ayxan
Tural
```

Kodun hissələri nə deməkdir?

```
open("participants.txt", ...)
```

Bu adla faylı açır.

Buradakı yol nisbi yoldur: fayl proqramın başladıldığı cari folderdə yaradılır. Bu folder həmişə Python faylının yerləşdiyi folder olmaya bilər. Bir az sonra yolu necə dəqiq müəyyən edəcəyimizi görəcəyik.

```
"w"
```

Faylı yazma modunda açır.

Fayl yoxdursa yaradır. Fayl artıq varsa, əvvəlki məzmununu silir.

```
encoding="utf-8"
```

Mətnin necə saxlanılacağını müəyyən edir. Azərbaycan hərflərini düzgün yazmaq və oxumaq üçün istifadə edirik.

```
as file
```

Açılmış faylla işləmək üçün file adlı variable yaradırıq. Sonrakı sətirlərdə həmin variable vasitəsilə fayla yazırıq.

```
file.write(...)
```

Verilən mətni fayla yazır.

```
with
```

Faylla işimiz bitəndə onu avtomatik bağlayır.

İçəri çəkilmiş sətirlər with blokuna aiddir. Fayla yazan sətirləri bu blokun daxilində saxlayın.

3. \n nə üçündür?

write() özü yeni sətərə keçmir.

Bu kodu yazsaq:

```
with open("participants.txt", "w", encoding="utf-8") as file:
    file.write("Ayxan")
    file.write("Tural")
```

Faylda belə görünəcək:

```
AyxanTural
```

Adların ayrı sətirlərdə olması üçün \n əlavə edirik:

```
with open("participants.txt", "w", encoding="utf-8") as file:
    file.write("Ayxan\n")
    file.write("Tural\n")
```

\n yeni sətirə keçid simvoludur. Faylda adi \n yazısı kimi görünmür; mətnin növbəti sətirə keçməsinə təmin edir.

4. w və a arasındakı fərq

w - məzmunu yenidən yazmaq

Faylda hazırda bunlar var:

```
Ayxan
Tural
```

Bu kodu run etsək:

```
with open("participants.txt", "w", encoding="utf-8") as file:
    file.write("Şəbnəm\n")
```

Əvvəlki adlar silinəcək. Faylda yalnız bu qalacaq:

```
Şəbnəm
```

Vacib: w rejimi əvvəlki məzmunu fayl açılan anda silir.

a - sona əlavə etmək

Faylda yenə Ayxan və Tural olduğunu düşünək.

```
with open("participants.txt", "a", encoding="utf-8") as file:
    file.write("Şəbnəm\n")
```

Nəticə:

```
Ayxan
Tural
Şəbnəm
```

Əvvəlki məzmun saxlandı, yeni ad sona əlavə olundu.

Bu kodu bir dəfə də run etsək, Şəbnəm yenidən əlavə ediləcək. Fayl özü təkrarlanan məlumatları yoxlamır.

Rejimlərə qısa baxış

Rejim	Məqsəd	Fayl yoxdursa
r	Oxumaq	Xəta yaranır.
w	Əvvəlki məzmunu əvəz edib yazmaq	Fayl yaranır.
a	Mövcud məzmunun sonuna əlavə etmək	Fayl yaranır.
x	Yalnız yeni fayl yaratmaq	Fayl yaranır; artıq varsa xəta yaranır.

5. Fayldan məlumatı necə oxuyuruq?

Əvvəl bu kodla faylı hazırlayın:

```
with open("participants.txt", "w", encoding="utf-8") as file:
    file.write("Ayxan\n")
    file.write("Tural\n")
```

Sonra oxuyun:

```
with open("participants.txt", "r", encoding="utf-8") as file:
    content = file.read()

print(content)
```

Nəticə:

```
Ayxan
Tural
```

Burada:

- r faylı oxumaq üçün açır;
- read() bütün məzmunu oxuyur;
- oxunan mətn content variable-ında saxlanılır;
- print(content) həmin mətni ekrana göstərir.

read() **nəticəni string olaraq qaytarır**. Faylda bir neçə sətir olması nəticəni avtomatik list etmir.

Bir sətir oxumaq: readline()

```
with open("participants.txt", "r", encoding="utf-8") as file:
    first_line = file.readline()
    second_line = file.readline()

print(first_line.rstrip("\n"))
print(second_line.rstrip("\n"))
```

Birinci çağırış Ayxan olan sətiri, ikinci çağırış Tural olan sətiri oxuyur.

`rstrip("\n")` oxunan sətirin sonundakı yeni sətir simvolunu çıxarır.

Sətirləri list kimi almaq: `readlines()`

```
with open("participants.txt", "r", encoding="utf-8") as file:
    lines = file.readlines()

print(lines)
```

Nəticə:

```
["Ayxan\n", "Tural\n"]
```

Sətirləri loop ilə oxumaq

```
with open("participants.txt", "r", encoding="utf-8") as file:
    for line in file:
        print(line.rstrip("\n"))
```

Hər dövrdə növbəti sətir oxunur.

6. Niyə ikinci `read()` boş nəticə verir?

```
with open("participants.txt", "r", encoding="utf-8") as file:
    first_read = file.read()
    second_read = file.read()

print(first_read)
print(repr(second_read))
```

Birinci `read()` bütün faylı oxuyur. Bundan sonra oxuma mövqeyi faylın sonundadır.

İkinci çağırışda oxunacaq əlavə mətn qalmadığı üçün nəticə boş string olur:

```
""
```

Fayl silinməyib. Sadəcə sonuna qədər oxunub.

Başlanğıca qayıtmaq üçün:

```
with open("participants.txt", "r", encoding="utf-8") as file:
    print(file.read())

    file.seek(0)

    print(file.read())
```

`seek(0)` oxuma mövqeyini faylın əvvəlinə qaytarır.

7. Faylın harada yaranacağını müəyyən etmək

Məlumat fayllarını layihəmizin içində saxlamaq istəyirik.

Bunun üçün Path istifadə edə bilərik:

```
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent
DATA_DIR = BASE_DIR / "data"

DATA_DIR.mkdir(exist_ok=True)

FILE_PATH = DATA_DIR / "participants.txt"
```

Addım-addım izah

`Path(__file__)`

Hazırkı Python faylının yolunu götürür.

`.resolve().parent`

Həmin faylın yerləşdiyi folder-in tam yolunu müəyyən edir.

`BASE_DIR / "data"`

Həmin folder-in içindəki data folder-inin yolunu qurur. Buradakı `/` yol hissələrini birləşdirir.

`mkdir(exist_ok=True)`

data folder-ini yaradır. Folder artıq varsa, buna görə xəta vermir.

`DATA_DIR / "participants.txt"`

Mətn faylının tam yolunu qurur.

Sonra faylı belə açə bilərik:

```
with open(FILE_PATH, "w", encoding="utf-8") as file:
    file.write("Ayxan\n")
    file.write("Tural\n")
```

Bu nümunədə data folder-i Python faylının yanında yaranır.

Module-unuz package daxilindədirsə, layihənin əsas folder-inə çatmaq üçün `.parent.parent` lazım ola bilər. Yolu öz strukturunuza uyğun seçin.

Fayl varmı?

```
print(FILE_PATH.exists())
```

Fayl varsa `True`, yoxdursa `False` qaytarır.

Bu yoxlama ilk run-da hələ yaranmamış faylı oxumağa çalışmamaq üçün faydalıdır.

8. Kitab list-ini niyə adi mətn kimi saxlamırıq?

Kitablarımız list daxilində dictionary-lər şəklindədir.

Adi mətn faylını `read()` ilə oxuyanda isə string alırıq.

Məsələn, faylda belə bir sətir olsun:

```
1984 | George Orwell | 1949
```

Python bu sətiri oxuyanda hansı hissənin kitab adı, hansı hissənin müəllif, hansı hissənin il olduğunu özü müəyyən etmir.

Bunun üçün məlumatı uyğun strukturda saxlamaq lazımdır. Bu homework-da **JSON** istifadə edəcəyik.

JSON məlumatları strukturla saxlayan mətn formatıdır. Python-un `json` module-u həmin məlumatları yazmağa və geri oxumağa kömək edir.

9. JSON-a yazmaq və JSON-dan oxumaq

Əvvəl dərsdəki kimi iki iştirakçı yaradaq:

```
participants = [  
    {"name": "Ayxan", "age": 24},  
    {"name": "Tural", "age": 19}  
]
```

Fayla saxlamaq

```
import json  
  
with open("participants.json", "w", encoding="utf-8") as file:  
    json.dump(  
        participants,  
        file,  
        ensure_ascii=False,  
        indent=4  
    )
```

Burada:

- `participants` - saxlayacağımız list;
- `file` - açılmış fayl;
- `json.dump()` - list-i JSON formatında fayla yazır;
- `ensure_ascii=False` - Azərbaycan hərflərinin birbaşa görünməsini təmin edir;
- `indent=4` - məzmunu oxunaqlı göstərir.

Faylın içi belə görünəcək:

```
[
  {
    "name": "Ayxan",
    "age": 24
  },
  {
    "name": "Tural",
    "age": 19
  }
]
```

Fayldan bərpa etmək

```
with open("participants.json", "r", encoding="utf-8") as file:
    loaded_participants = json.load(file)

print(loaded_participants)
print(loaded_participants[0]["name"])
```

Sonuncu nəticə:

Ayxan

`json.load()` burada yenidən list qaytarır. List-in içindəki məlumatlar da dictionary olaraq bərpa olunur.

Yaş dəyərləri rəqəm olaraq saxlanıldığı üçün oxunanda integer olaraq qalır.

Funksiyaları qarışdırmayın

Funksiya	Nə ilə işləyir?
<code>json.dump(data, file)</code>	Məlumatı fayla yazır.
<code>json.load(file)</code>	Məlumatı fayldan oxuyur.
<code>json.dumps(data)</code>	Məlumatı JSON string-ə çevirir.
<code>json.loads(text)</code>	JSON string-i Python məlumatına çevirir.

Faylla işləyən variantlarda `s` yoxdur.

Vacib: Bütün list-i saxlayarkən fayla onun cari vəziyyətini yazın. Tam list-ləri `a` ilə bir-birinin ardınca əlavə etmək normal tək JSON sənədi yaratmır.

10. Oxunan məlumatı proqramda istifadə etmək

Fayldan məlumatı oxumaq hələ həmin məlumatın proqramınızda istifadə edilməsi demək deyil.

Oxunan list kitab əməliyyatlarının işlədiyi list-ə yerləşdirilməlidir.

Dərsdə bunu belə etmişdik:

```
participants.clear()
participants.extend(loaded_participants)
```

- `clear()` mövcud list-in içini boşaldır.
- `extend()` oxunan list-in elementlərini həmin list-ə əlavə edir.

Bu üsul mövcud list-in özünü saxlayaraq içindəki məlumatları yeniləyir.

Fayldan oxuyan funksiya nəticəni başqa hissəyə ötürmək üçün `return` istifadə etməlidir. `print()` isə yalnız ekrana göstərir.

11. Tarix və saatı mətn kimi saxlamaq

Əməliyyatın vaxtını qeyd etmək üçün:

```
from datetime import datetime

operation_time = datetime.now().isoformat(timespec="seconds")

print(operation_time)
```

Nəticə belə ola bilər:

```
2026-10-09T19:30:00
```

- `datetime.now()` hazırkı tarix və saatı götürür.
- `isoformat()` onu string-ə çevirir.
- `timespec="seconds"` saniyəyə qədər göstərir.

Bu string-i mətn faylına və ya JSON məlumatına əlavə edə bilərsiniz.

Tapşırıqlar

Layihə: Book Management System-in davamı

Lesson 11-də hazırladığınız layihəyə aşağıdakı imkanları əlavə edin.

Əvvəlki kitab əlavə etmə, göstərmə, axtarış, silmə və statistika funksionallıqlarını qoruyun.

Faylların və funksiyaların adlarını, yeni module-ların yerini özünüz müəyyən edin.

1. Kitabları JSON faylında saxlayın

Sistemdəki kitab list-i bir JSON faylında saxlanılmalıdır.

Hər kitabın aşağıdakı məlumatları qalmalıdır:

- kitabın adı;
- müəllif;
- buraxılış ili.

Gözlənilən davranış

Sistemdə 3 kitab varsa və yeni kitab əlavə edirsinizsə, saxlanılan faylda həmin 4 kitabın hamısı olmalıdır.

Bir kitabı silirsinizsə, faylda qalan 3 kitab saxlanılmalıdır.

Tələblər

- Uğurlu əlavə etmədən sonra dəyişiklik fayla yazılmalıdır.
- Uğurlu silmədən sonra fayl yenilənməlidir.
- Əvvəlki kitablar təsadüfən itirilməməlidir.
- Azərbaycan hərfləri düzgün saxlanılmalıdır.
- JSON məzmunu oxunaqlı olmalıdır.
- Buraxılış ili integer olaraq saxlanılmalıdır.

2. Proqram başlayanda kitabları fayldan oxuyun

Proqram əvvəlcə saxlanılmış kitabları bərpa etməlidir.

Gözlənilən davranış

Bu gün 5 kitab əlavə edib proqramı bağlayırsınız.

Proqramı yenidən başladanda həmin 5 kitab sistemdə görünməlidir. Onları yenidən kod vasitəsilə əlavə etməyə ehtiyac olmamalıdır.

Tələblər

- JSON faylı varsa, kitablar ondan oxunmalıdır.
- Fayl yoxdursa, sistem boş kitab list-i ilə başlamalıdır.
- Bərpa olunan kitablar axtarış, silmə və statistika əməliyyatlarında istifadə edilməlidir.
- Təkrar run kitab sayını artırmamalıdır.
- Silinmiş kitab növbəti run-da yenidən yaranmamalıdır.

Diqqət: Hər run-da eyni test kitablarını şərtsiz əlavə etməyin.

3. Mətn hesabatı yaradın

Kitab siyahısını insanın rahat oxuya biləcəyi .txt faylına çıxarın.

Hesabatda bunlar olmalıdır:

- kitabın adı, müəllifi və ili;
- ümumi kitab sayı;
- 2000-ci ildən əvvəl çıxan kitabların sayı;
- orta buraxılış ili.

Nümunə:

```
--- Kitab siyahısı ---

1984 | George Orwell | 1949
Animal Farm | George Orwell | 1945
The Hobbit | J.R.R. Tolkien | 1937
Clean Code | Robert C. Martin | 2008
Python Crash Course | Eric Matthes | 2015

Ümumi kitab sayı: 5
2000-ci ildən əvvəl çıxan kitabların sayı: 3
Orta buraxılış ili: 1970.8
```

Tələblər

- Hər kitab ayrıca sətirdə göstərməlidir.
- Hesabat yenidən yaradıldıqda əvvəlki hesabat əvəz edilməlidir.
- Eyni siyahı faylın sonuna təkrar əlavə olunmamalıdır.
- Kitab yoxdursa, hesabat bunu bildirməlidir.
- Boş list üçün orta il hesablanarkən xəta yaranmamalıdır.

Hesabatın görünüşünü özünüz müəyyən edə bilərsiniz.

4. Hesabatı fayldan oxuyub göstərin

Əvvəlki tapşırıqda yaratdığınız .txt hesabatını oxuyub ekrana göstərin.

Tələblər

- Məlumat həqiqətən hesabat faylından oxunmalıdır.
- Hesabat yoxdursa, onun hələ yaradılmadığını bildirin.
- Oxuma funksiyası məzmunu qaytarmalıdır.
- Qaytarılmış məzmun proqramın uyğun hissəsində ekrana göstərməlidir.

Burada kitab list-indən yeni hesabat hazırlamaqla mövcud hesabat faylını oxumaq arasındakı fərqə diqqət edin.

5. Proqramı ayrı run-larda yoxlayın

Birinci run

1. Ən azı 5 kitab əlavə edin.
2. Onların arasında eyni müəllifə aid ən azı 2 kitab olsun.
3. Ən azı 2 kitab 2000-ci ildən əvvəl, 2 kitab isə 2000-ci ildən sonra çıxmış olsun.
4. Kitablari saxlayın.
5. Mətn hesabatı yaradın.
6. Proqramı bitirin.

İkinci run

1. Kitablari yenidən əlavə etmədən JSON faylından bərpa edin.
2. Kitab sayının 5 olduğunu göstərin.
3. Bir kitabı adına görə axtarın.
4. Bir müəllifə aid kitablari axtarın.
5. Bir kitabı silin.
6. Dəyişikliyi saxlayın və proqramı bitirin.

Üçüncü run

1. Kitablari yenidən fayldan bərpa edin.
2. Kitab sayının 4 olduğunu göstərin.
3. Silinmiş kitabın artıq tapılmadığını yoxlayın.
4. Hesabatı yeniləyin.
5. Hesabatı fayldan oxuyub ekrana göstərin.

Bu yoxlama məlumatların proqram bağlandıqdan sonra da saxlanıldığını göstərməlidir.

Final Task - Əməliyyat tarixçəsi

Kitab əlavə etmə və silmə əməliyyatlarını ayrıca .txt faylında qeyd edin.

Hesabat cari kitab siyahısını göstərir. Tarixçə isə əvvəl hansı əməliyyatların edildiyini saxlayır.

Hər uğurlu əməliyyat üçün yeni sətirdə bunlar yazılmalıdır:

- tarix və saat;
- əməliyyatın növü;
- kitabın adı.

Nümunə:

```
2026-10-09T19:30:00 | ƏLAVƏ EDİLDİ | 1984
2026-10-09T19:35:00 | ƏLAVƏ EDİLDİ | The Hobbit
2026-10-09T19:40:00 | SİLİNDİ | 1984
```

Tələblər

- Yeni əməliyyat əvvəlki sətirləri silməməlidir.
- Əməliyyatlar faylın sonuna əlavə edilməlidir.
- Proqram yenidən başladıqda əvvəlki tarixçə qalmalıdır.
- Kitablara fayldan bərpa etmək yeni kitab əlavə etmə kimi qeyd olunmamalıdır.
- Düzgün olmayan kitab məlumatı qəbul edilməyibsə, uğurlu əlavə etmə qeydi yazılmamalıdır.
- Silinmək istənilən kitab tapılmayıbsa, uğurlu silmə qeydi yazılmamalıdır.

Hansı fayl rejiminin uyğun olduğunu özünüz müəyyən edin.

Tamamladıqdan sonra layihənin GitHub'a commit edin..